

ACOL 226

6th April

(Church)	<u>Functional Programming</u>	5 hours
	<u>The lambda calculus</u>	7 hours
	<u>Logic programming</u>	3 hours

A functional program is a collection of functions that work together to transform data.

1977

Turing Award winning lecture
by John Backus.

Haskell

C, C++, Java
Imperative

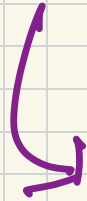
programs are a sequence of commands
describe how to manipulate the state. ← that

Imperative programming languages
are closely related with the way
modern computers are designed.

von Neumann architecture

The way of describing computation as a
sequence of instructions (that update individual
memory locations)
"hides" the structure of the overall
problem.

This makes it difficult to decompose large systems / programs into smaller functional blocks that could be put together efficiently and correctly.



Functional programming (Haskell)

↳ the way of defining a function in Haskell is to simply "declare" what it does.

1930's → Church gave the
abstract model of computation
called the lambda calculus.
↳ provided a notation for describing
functions that could be effectively
computed

LISP

1950's

↳

list programming (John McCarthy)

↳

no notion of data type

values in Lisp are treated as
abstract symbols

↳

symbolic computation

ML

typed programming language

1980's

Haskell

unified different
features from several
functional programming languages.

Haskell

Programs in Haskell are functions that transform inputs to outputs.

↳ The description of these functions has two parts:

✓ i)

the types of inputs and outputs

✓ ii)

the rule for computing the output from the input.

$$\text{square}(x) = x^2$$

$$\text{square} : \mathbb{Z} \rightarrow \mathbb{Z}$$

$$\text{square} : \mathbb{R} \rightarrow \mathbb{R}$$

$$\text{square} : \mathbb{C} \rightarrow \mathbb{C}$$

$$f :: \text{A} \rightarrow \text{B}$$

$$\checkmark \underline{\text{square}} :: \underline{\text{Int}} \rightarrow \underline{\text{Int}}$$

$$\checkmark \underline{\text{square}} \quad \underline{x} = \underline{x * x}$$

Haskell has basic built-in scalar types

Int, Float, Bool, Char

In addition to this, given a type T ,
Haskell allows you to form a
list of elements of type T .

The list has
the type $[T]$

factorial  Int $\rightarrow$ Int

$\text{max} :: \text{Int} \times \text{Int} \rightarrow \text{Int}$

where this $(\text{Int} \times \text{Int})$
denotes the set of all
pairs of integers
type Int

$(_ \times _ \times _ \dots \times _)$

$f: T_1 \times T_2 \times T_3 \times \dots \times T_n \rightarrow T$
return type

Haskell bypasses the need to store the function arity by assuming that each function takes only one argument.

$\text{max} :: \underline{\text{Int}} \rightarrow (\text{Int} \rightarrow \text{Int})$

$(\text{max } 2) 3$
↓

max-of-2-and

$\underline{n} = n \text{ if } n > 2$
 $= 2 \text{ otherwise}$

plus 2 3

plus :: Int → (Int → Int)

plus 2 3

plus 2 :: Int → Int
plus 2 n = n + 2

$$f(x_1, x_2, x_3, \dots, x_n)$$

$$\underbrace{\underbrace{((f\ x_1)\ x_2)\ x_3)\ x_4)\ x_5 \dots}}_{\text{currying}}$$

$$f :: \underline{T_1} \rightarrow \underline{\underline{(T_2 \rightarrow (T_3 \rightarrow (T_4 \dots (T_n \rightarrow T)))))}}$$

This convention by which all functions are regarded as functions taking a single argument is called currying → Haskell Curry